Table 4.3 Running time of operations on Dictionaries, for different implementations

Operation	Unsorted Arrays	Sorted Arrays	BST	Hash Table
Create the DS	O(1)	O(n*log(n))	O(n*log(n))	O(n)
Look-up an entry	O(n)	O(log(n))	O(log(n))	O(n/M) (§)
Add a new entry	O(1) (§)	O(n)	O(log(n))	O(n/M) (§)
Remove an entry	O(1)	O(n)	O(log(n))	O(n/M) (§)
Sorted List	O(n*log(n)	O(n)	O(n)	O(M+n*log(n))
Min/Max	O(n)	O(1)	O(1) (*)	O(M+n)
Prev/Next	O(n)	O(1)	O(log(n))	O(M+n)

(*) By storing max/min separately, and amortized the time to replace them on insert/delete.

Looking at table 4.3 it's evident that, if we don't have to worry about any operation that involves the order of the element, or the order they were inserted, then the amortized time of hash-tables is the best: if n~=M (and so there are approximately as many buckets as elements), then an hash table can perform insertion, removal and look-up in amortized constant time.